SPECIAL PROJECT · DRAFT FOR DISCUSSION

Agent OS

A system-level AI assistant on a GrapheneOS fork

Prompt the phone. Drive any app. Keep the model where you choose: on the device, on your own GPU, or, if you opt in, in the cloud.

Mohammed Husari · 5 November 2026

WHAT WE WANT

One assistant that can do anything you can do on the phone

1 Prompt, don't tap

"Reply to Ali that I'm 10 minutes late." "Order my usual." "What did I miss on Signal and WhatsApp?" Voice or text, with a wake word that works with the screen off.

2 Every app, not a chosen few

Messaging, email, maps, shopping, delivery, banking (read-only), browser: the agent reads any screen and acts in any app, with no per-app integration.

3 Local-first and private

Models run on the phone or on a GPU you own. Nothing leaves the device unless you allow it, per app and per role. Cloud is an option, off by default.

Non-negotiable

- **Full control of the OS and device.** Nobody else decides what the agent may see or do.
- **Safe against hostile content.** A message or web page must never be able to trigger an action by itself.

Explicitly not a goal

- Faking a different device to pass bank or app security checks.
- Automating logins, payments or identity verification without a real touch.

WHY NOT JUST BUILD AN APP?

An app runs out of permissions long before it runs out of ideas

CAPABILITY	NORMAL APP ON GRAPHENEOS	PRIVILEGED SYSTEM COMPONENT (OUR FORK)
Read any screen	**Accessibility service only. Hidden "sensitive" views, blocked by banks, discouraged by GrapheneOS**	**Yes, via WindowManager, including FLAG_SECURE screens**
Tap, swipe, type in other apps	**Accessibility gestures; detectable, fragile**	**INJECT_EVENTS: indistinguishable from a finger**
Wake word with screen off	**No. Needs privileged audio permissions**	**SoundTrigger HAL, low-power audio chip**
Never killed in background	**No. Battery management stops it**	**Persistent system process**
Direct access to app data	**No**	**Dedicated SELinux domain (with Keystore caveats)**
Enforce policy the agent itself can't bypass	**No. The app polices itself**	**Policy and confirmation live in system_server**
Depends on Google's DMA APIs (2027+)	**Yes, and they may be EU-only and certification-gated**	**No**

Gemini (Google) and Doubao (ByteDance + ZTE) both work only because they ship as privileged system software. There is no app-level route to the same thing today.

THE LANDSCAPE

Everyone with a system-level agent is a platform owner

Gemini screen automation (Google)

Beta since Feb 2026 on Galaxy S26 and Pixel 10. Multi-step tasks in Uber, DoorDash, Grubhub. US and Korea only, selected apps, cloud model. Works because Gemini is privileged Google system software.

Doubao Phone Assistant (ByteDance + ZTE Nubia)

Dec 2025, Nubia M153, China only, ~$495, sold out. ZTE's Android granted ByteDance's agent INJECT_EVENTS. Within days WeChat, Alipay, Taobao and banks logged users out ("abnormal login environment"). WeChat control withdrawn 3 Dec.

EU DMA decision (16 July 2026)

Google must give third-party assistants equal access to 11 features Gemini uses, incl. screen automation and on-device models. Deadline Android 18 / 1 Aug 2027. Five features may need certification. Unclear if it reaches AOSP.

App-level tools

Tasker (automation + AI generator, MCP servers), **gos-ai** (GrapheneOS-targeted assistant), **Liquid Apollo / LEAP** (on-device LFM models), **Omarchy mobile** (Linux replacing Android). Useful parts, none has system privileges.

Takeaway: the agent is the easy part; the privileges are the hard part. Google grants them to itself, ZTE granted them to ByteDance. We grant them to ourselves by owning the OS.

OPTIONS CONSIDERED

Five ways to get there, one gives us control now

APPROACH	CONTROL	AVAILABLE	KEEPS HARDENING	MAIN COST
A. App on stock GrapheneOS Assistant role, notifications, accessibility	Limited	Now	Yes	Hits the permission ceiling (slide 3)
B. Wait for Google's DMA APIs Certified assistant, Android 18	Partial	2027+, EU?	Stock Android	Google sets the rules, timeline and gatekeeping
C. Linux phone Omarchy-mobile / postmarketOS	Total	Immature	No	Lose Android apps, calls/SMS unverified
D. Partner with a phone maker The ByteDance + ZTE model	Negotiated	Years	Vendor Android	Need a manufacturer; they own the keys
E. Fork GrapheneOS on Pixel ← chosen Agent as a privileged system service	Total (OS)	Now	Yes, inherited	Rebase onto every GrapheneOS release; own signing key

A remains the fallback for stock-Android users and B a possible second distribution route from 2027. Neither delivers full control today.

WHY THIS APPROACH

Fork GrapheneOS: we hold the ZTE role and the ByteDance role at once

- **Full control of the OS layer.** Platform signing key, system services, permissions, SELinux policy. Nobody to ask.
- **Hardening for free.** GrapheneOS's memory tagging, exploit mitigations, sandboxed Google Play, per-profile encryption all come with the base. We only add the agent.
- **Pixel hardware, no hardware project.** Titan M2, Tensor TPU, verified boot with our own key, relockable bootloader.
- **Local-first by construction.** GrapheneOS ships no AI and no telemetry. Every model byte path is ours to define.
- **Safety enforced below the agent.** Policy and confirmation live in system_server, so even our own agent (or a prompt injection driving it) cannot bypass them. Only an OS owner can build this.

Where we stand relative to Doubao / ZTE

LAYER	DOUBAO PHONE	US
Agent app	ByteDance	**Ours**
OS granting privileges	ZTE's Android	**Our GrapheneOS fork**
Model	ByteDance cloud	**Device / home GPU / opt-in cloud**
Firmware, modem	ZTE	**Google's, closed**
Attestation identity	Known OEM key	**Our key, unknown to banks**

Everything the agent needs lives in the OS layer. Firmware control is the one thing a manufacturer has that we don't, and the agent doesn't need it.

SUMMARY SPEC · ARCHITECTURE

What we add to the OS

APPS	Signal · WhatsApp · Telegram · Mail · Maps · Shopping · Bank (read-only) · Browser · **Agent app** privileged, platform-signed, never killed: voice, planner, skills library
ADDED BY US	**AgentManagerService**: the only path to every power below. Holds per-app policy ("Signal: read + draft", "Bank: notifications only"), model routing rules, action log. **Trusted confirmation**: SystemUI dialog that accepts real touches only, for send / pay / delete / install.
FRAMEWORK	WindowManager + secure screen capture · UI tree (view hierarchy) · InputManager · INJECT_EVENTS · SoundTrigger HAL · hotword · Notifications · Telephony · SMS · Intents · deep links · AppFunctions
SECURITY	SELinux: agent domain, app-data read where granted · Keystore (Signal DB key stays out of reach without changes) · TPU allowlist: agent added alongside Google apps · Verified boot with our key
HARDWARE	Pixel 9 Pro / 10 Pro (16 GB RAM) · Tensor TPU · Titan M2 · low-power audio DSP

Seeing: UI tree when good, screenshot fallback for Flutter / React Native / WebView / games. Acting: injected input, our own system keyboard for typing, intents where they exist. Successful tasks are saved as replayable per-app **skills**, so the model is only needed for new tasks.

SUMMARY SPEC · MODEL ROUTING

Interchangeable brains, user-controlled, enforced by the OS

Backends

- **On-device** (always): Tensor TPU / GPU, LEAP or llama.cpp, 3–8B models.
- **Home server** (optional): your GPU over WireGuard, vLLM or llama.cpp server, OpenAI-compatible API so models swap without touching the phone.
- **Cloud** (optional, **off by default**).

Roles, each routed separately

Wake word, speech-to-text	**Device only, locked**
Reader (untrusted content)	Device, changeable
Screen understanding	Home → device fallback
Planner (next action)	Home → device fallback

Rules in AgentManagerService

- **Per app:** "banking and Signal screens never leave the phone" wins over everything.
- **Per connection:** home server on home Wi-Fi, optionally on fast mobile data.
- **Fallback:** unreachable server → on-device, task continues.
- **Status-bar indicator:** phone / home / cloud, like the mic and camera dots.

<1 s

Typical screen-read step on the home GPU over Wi-Fi; several seconds for a vision model on the phone

0

Model calls needed to replay a saved skill: steps replay, each screen is checked

1

Extra device to secure: the home server sees whatever screens you route to it

SUMMARY SPEC · SAFETY

Full power, but the agent can never approve its own actions

1 Trusted confirmation

Send, pay, delete, post and install route through AgentManagerService, which shows a SystemUI prompt that **ignores injected input**. A malicious message can steer the model; it cannot press the button.

2 Planner / reader split

The planner sees labels ("3 new Signal messages from Ali") and is the only part that can call tools. The reader sees raw content and can only summarise or draft, so injected text ends up in a summary, never in an action.

3 Per-app grants and an action log

One policy store, one screen, enforced in the OS. Every action is logged. Biometric prompts stay in secure hardware, so real payments still need you.

Lessons from Doubao

- **The tech worked; the ecosystem pushed back.** WeChat, Alipay and banks detected the agent server-side and logged users out. A famous product on one phone model is an easy target.
- **Human-like pacing** on taps and reads, so behaviour doesn't read as a script.
- **Don't automate logins, payments or identity checks.** Doubao withdrew these after the backlash; we exclude them from day one.
- **Every app must still work by hand.** If one objects, we stop automating it. We do not disguise the device or agent to defeat an app's security checks.
- **Shipping to others changes the picture.** A personal fork is invisible; a popular agent phone gets the full Doubao treatment.

KNOWN LIMITS AND RISKS

What the fork costs, and what still can't be driven

Banking apps and attestation

Behaviour checks: we pass (no accessibility service, native input). Hardware attestation: Titan M2 reports *our* signing key, which no bank recognises. We lose only the banks that specifically accept GrapheneOS's key; apps that check nothing keep working, apps that need Play Integrity already fail on GrapheneOS.

BANK CHECKS	GRAPHENEOS	OUR FORK
Nothing (most)	**Works**	**Works**
Accepts GrapheneOS key	**Works**	**Fails**
Play Integrity device	**Fails (~10%)**	**Fails**

Update burden

GrapheneOS releases often. Every release means rebase, build, sign, OTA. This is an ongoing job, to be automated with a capability test suite (~20 apps) run on every build.

Still out of reach

Biometric prompts, CAPTCHAs and liveness checks, DRM video (black even to the OS), data that never reaches the phone, server-side bans for bot-like behaviour.

Model capability

Open screen-reading models (UI-TARS, Qwen-VL, AgentCPM-GUI) handle short tasks in known apps; long tasks across unfamiliar apps break. Skills library and home GPU are the mitigation.

WHAT WE INTEND TO DO

Five phases, starting with a build we sign ourselves

1

Own the pipeline

Build stock GrapheneOS from source, generate signing keys (offline storage), flash a Pixel, relock the bootloader with our key, run our own OTA server. No custom code yet.

WEEKS 1–2

2

Agent v0

AgentManagerService + privileged agent app. Screen capture, UI tree, INJECT_EVENTS, notifications, on-device model. Test: drive Signal, WhatsApp, Telegram end to end by voice.

WEEKS 3–8

3

Safety in the OS

Trusted confirmation dialog, per-app policy store, planner/reader split, action log. Model routing: home GPU over WireGuard with device fallback and status indicator.

WEEKS 9–12

4

Any app

Wake word via SoundTrigger, telephony, system keyboard, skills library. Test set of ~20 apps across categories incl. a Flutter app, a React Native app and a game.

MONTHS 4–5

5

Sustain

Automated rebase onto each GrapheneOS release with the capability suite as gate. Decide on distribution (personal vs. others) and on a 2027 DMA-API app route for stock Android.

ONGOING

Needs: AOSP build machine (64 GB+ RAM, ~500 GB SSD, cloud is fine) · 1–2 engineers with Android platform experience (system services, SELinux, Soong/AIDL) · 2–3 Pixels · home GPU (24 GB+ VRAM) · offline key storage.

OPEN DECISIONS

What we need to settle before Phase 1

1 Audience

Personal device only, a product for GrapheneOS users, or a relationship / upstream play with the GrapheneOS team? Sets polish, licence (open source for the last two), and whether the Doubao-style pushback is a real risk.

2 Hardware

Which Pixel (9 Pro vs 10 Pro, 16 GB) and which home GPU (24 GB vs 32 GB+ VRAM). Together they decide which models are realistic for planner and vision.

3 GrapheneOS team

Ask about: open-source STT/TTS timeline, planned accessibility safeguards, how they'll handle Android 18 DMA APIs if they reach AOSP, and whether they'd advise a fork on hardening and update cadence.

4 Which apps matter most

The daily jobs, the banking / fintech apps in use (check each against the attestation table), and what the "FT app" is: Financial Times or a fintech app.

Recommendation: start Phase 1 this week with one Pixel and a cloud build box. It costs little, proves the pipeline, and nothing in it is wasted whatever we decide on 1–4.